Fine Tuning LLMs

is basically defined as Tuning LLM model to our specific tasks, domains or styles by updating their weight with smaller targeted Datasets.

Types of Fine Tuning ;

1. **Full parameter Fine Tuning**
   - Train all weights
   - Maximum Accuracy
   - very costly
2. **PEFT based Fine Tuning**
   - Instead of Training all weights we only train a small subset
   - **LoRA** → Add small trainable matrices into layers.
   - **QLoRA** → LoRA + Quantization.
3. **Quantization**

not fine tuning itself but compression technique.

- Usually combined with PEFT.

Q. what is Quantization ?

weights are saved in FP32

↓

Floating point 32

Full precision 32

Quantization: basically helps us save weights in int 8 bit rather than FP32

FP32 ⟶ 8 bit Integers
Lower Precision

Q. Why this works?

1. weights dont need extreme precision
2. Memory savings (Imp!!)

FP32 = 4 bytes
8 bit = 1 byte
{ 4x memory saving

3. Smaller Numbers → Less compute

working

Imp: Int8 = Round (FP32 / Scale)

Remember; we can downgrade FP32 → Int8 → Int4 but major con, precision will also decrease.

Q. Why is Full parameter Tuning a problem?

It is a problem because it takes lots of Resources even for Downstream tasks

→ Downstream Task = Monitoring, Inferencing, GPU, RAM

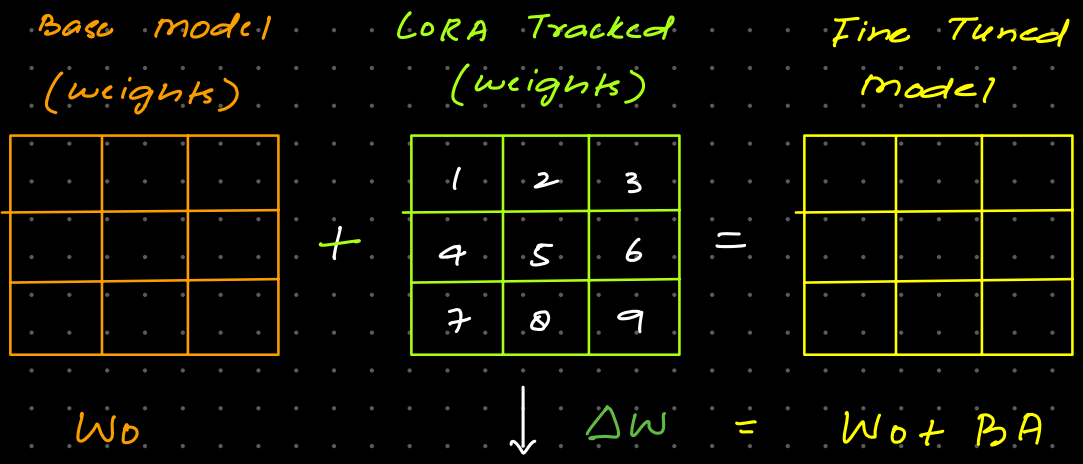
→ Updating all parameters means updating > 175 B parameter.

→ Hardware Resource Constraint

Q. LoRA → Matrix Decomposition

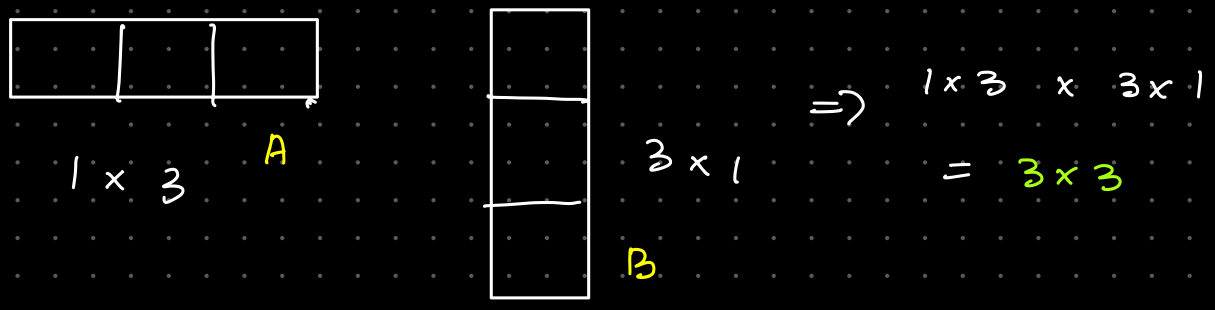
LoRA fine Tunes LLM models by Decomposing weight updates into lower Rank Matrices

→ Instead of updating all the weights it tracks changes



Imp.

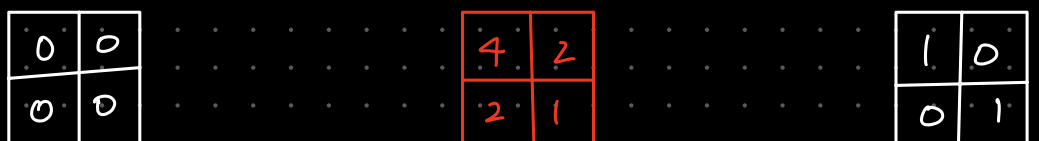
Rank (1) Matrix Decomposition



Hence, with LoRA Matrix Decomposition happens which means number of elements also Reduce.

Q. what is Rank in Matrix ?

Rank is based on Independence of elements in the Matrices



Why 1 Rank ?

Because 4 2, 2 1 is in the same linear Direction (like a single line)

Imp. ⇒ Q. How does it work ?

LoRA adapters are added in Attention Layers which means (QKV) like more likely words are being Targeted → That's how our model learns more towards (Domain Specific) making it fine tuned.

Yes, you've got the essence right ✅

- **LoRA attaches adapters to attention layers** (most commonly the linear projection matrices for **Q, K, V, and O** inside the transformer's attention blocks). These are the spots where small low-rank updates have the biggest impact.
- **Attention layers in LLMs** are the mechanism that decides *which tokens (words)* to focus on when generating the next output. They don't literally "prefer higher predictive words," but they **assign weights (attention scores)** to tokens based on relevance.
  - For example, in the sentence *"The stock market crashed because..."*, the attention mechanism will give more weight to "crashed" and "because" when predicting the next word, rather than irrelevant words like "the."
  - This is why fine-tuning adapters here is powerful: small changes in attention can shift the model's focus toward domain-specific vocabulary (like finance terms, medical jargon, or legal phrasing).

✅ **So in short:**

- LoRA adapters are usually attached to **attention layers**.
- Attention layers are the part of the model that **decide which words matter most** for prediction.
- By fine-tuning only these, you steer the model's focus toward your dataset (e.g., finance), while leaving the rest of the billions of parameters untouched.

🎯 Interview-Perfect Summary

> PEFT methods like LoRA are usually better than full fine-tuning because they train only a tiny fraction of parameters, reduce compute, prevent catastrophic forgetting, and allow multiple tasks through small adapters. Among PEFT methods, LoRA is the most widely used and the most stable choice.

Q. What is Prefix Tuning or P-Tuning?

✅ 1. Example of Prefix Embedding (Prefix Tuning)

This is where we *prepend trainable vectors to the input*.

Original input

P1 = Simple Tasks

```
"Summarize this article"
```

Tokenized (simplified):

```
[ w1, w2, w3 ]
```

After adding prefix (example with 4 prefix vectors)

```
[ P1, P2, P3, P4, w1, w2, w3 ]
```

Where:

- **P1–P4** are *trainable embeddings*
- They are **not real words**, but learned vectors
- These remain fixed length (e.g., 4, 10, 20 prefix tokens)

What the model sees

```
[p1] [p2] [p3] [p4] Summarize this article
```

This teaches the model task-specific behavior without touching its main weights.

✅ 2. Example of Prefix Layer (Deep Prefix / P-Tuning v2)

Here we add **prefix key/value vectors to attention** for each layer.

Let's assume a transformer with **3 layers**.

Layer 1 Attention KV becomes

P2 - Complex Tasks

```
[ P1_KV ] + [ Original_KV ]
```

Layer 2 Attention KV becomes

```
[ P2_KV ] + [ Original_KV ]
```

Layer 3 Attention KV becomes

```
[ P3_KV ] + [ Original_KV ]
```

So for each layer, you prepend **trainable KV pairs**.

A diagram:

```
Transformer Layer 1:
   Prefix_K → [Pk1_K]
   Prefix_V → [Pk1_V]
   Then attention happens on [Prefix + Original]

Transformer Layer 2:
   Prefix_K → [Pk2_K]
   Prefix_V → [Pk2_V]

Transformer Layer 3:
   Prefix_K → [Pk3_K]
   Prefix_V → [Pk3_V]
```

This gives richer conditioning than adding prefixes only to the input.

⭐ When to Use What — Short Table

Method	When to Use	Why
PEFT (LoRA / QLoRA)	Most fine-tuning tasks, instruction tuning, domain adaptation	Best performance–compute balance, stable, lowest GPU use
Prefix Embedding (P1)	Ultra-low compute, tiny datasets, simple tasks	Lightest method, fastest, minimal parameters
Prefix Layer (P2 / P-Tuning v2)	Need higher accuracy without full FT, large models, complex tasks	Injects task signals at every layer, more expressive than P1

Imp. → LoRA is usually best because

Method	Hyperparameters	# Trainable Parameters
Fine-Tune	-	175B
PrefixEmbed	$l_p = 32, l_i = 8$	0.4 M
	$l_p = 64, l_i = 8$	0.9 M
	$l_p = 128, l_i = 8$	1.7 M
	$l_p = 256, l_i = 8$	3.2 M
	$l_p = 512, l_i = 8$	6.4 M
PrefixLayer	$l_p = 2, l_i = 2$	5.1 M
	$l_p = 8, l_i = 0$	10.1 M
	$l_p = 8, l_i = 8$	20.2 M
	$l_p = 32, l_i = 4$	44.1 M
	$l_p = 64, l_i = 0$	76.1 M
Adapter^H	$r = 1$	7.1 M
	$r = 4$	21.2 M
	$r = 8$	40.1 M
	$r = 16$	77.9 M
	$r = 64$	304.4 M
LoRA	$r_v = 2$	4.7 M
	$r_q = r_v = 1$	4.7 M
	$r_q = r_v = 2$	9.4 M
	$r_q = r_k = r_v = r_o = 1$	9.4 M
	$r_q = r_v = 4$	18.8 M
	$r_q = r_k = r_v = r_o = 2$	18.8 M
	$r_q = r_v = 8$	37.7 M
	$r_q = r_k = r_v = r_o = 4$	37.7 M
	$r_q = r_v = 64$	301.9 M
	$r_q = r_k = r_v = r_o = 64$	603.8 M

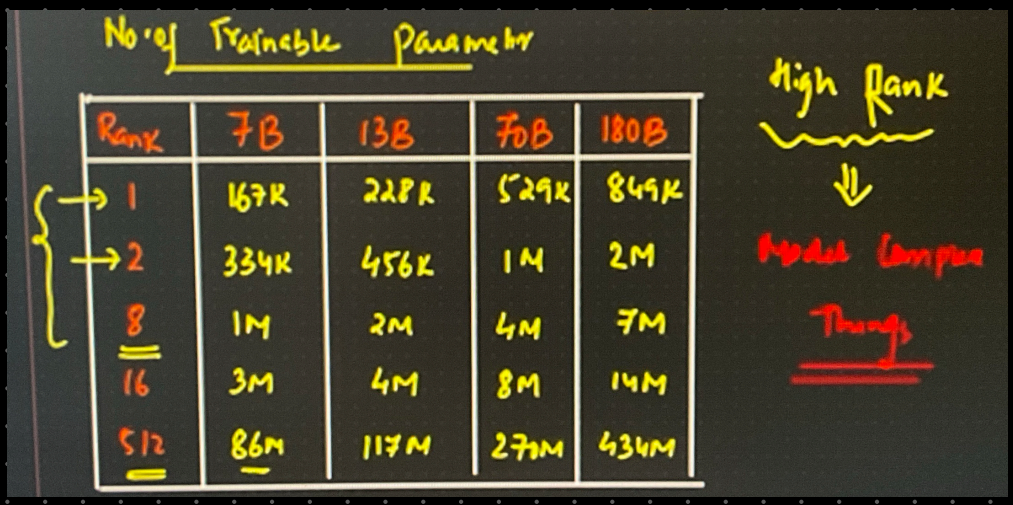
175Billion

7.1M

Matrix Dec

LoRA params are lowest.

Imp. For LLAMA 2, we always use Rank = 2.



Q. Q LoRA ?

is Quantized LoRA

So; we first convert

↳ weights into Quantized weights

→ fp32 → Int 8 or Int 4

↳ Then Fine Tune using LoRA Tech